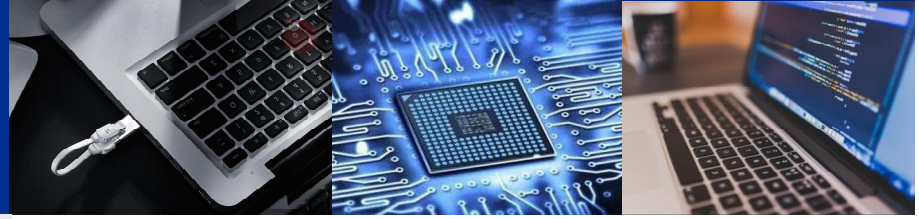


4 汇编语言程序设计

- 汇编语言程序基本格式
- 汇编语言中的数据
- 运算符与表达式
- 伪指令
- 系统功能调用
- 宏指令
- 汇编语言程序设计举例
- 汇编语言程序上机过程
- 调试程序DEBUG的使用



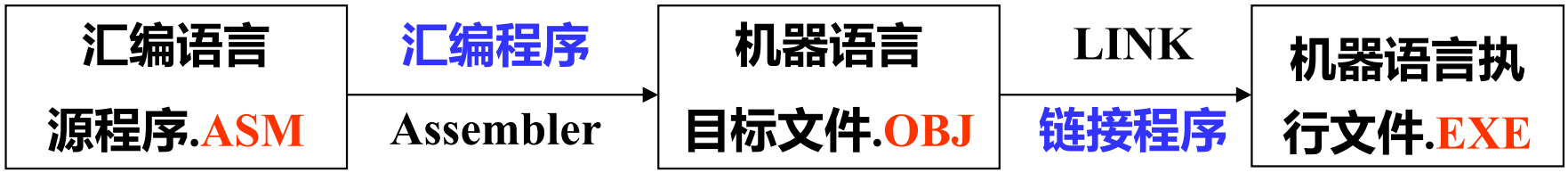
4.1汇编语言程序基本格式

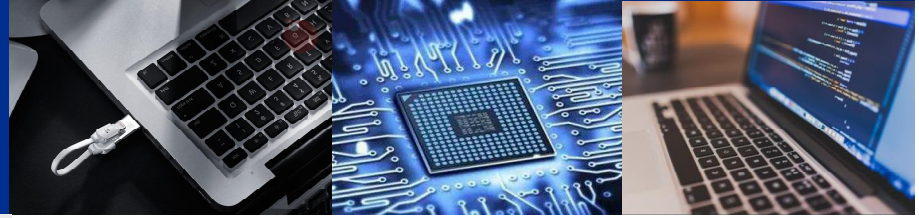
4.1.1汇编语言源程序和汇编程序

汇编程序以汇编语言源程序文件作为输入，并由它产生两种输出文件：目标程序文件和源程序列表文件。

- 目标程序文件经**链接定位**后由计算机执行；
- 源程序列表文件将列出源程序、目标程序的机器语言代码及符号表。

汇编语言源程序的处理过程如下：





4.1汇编语言程序基本格式

4.1.2汇编语言的特点

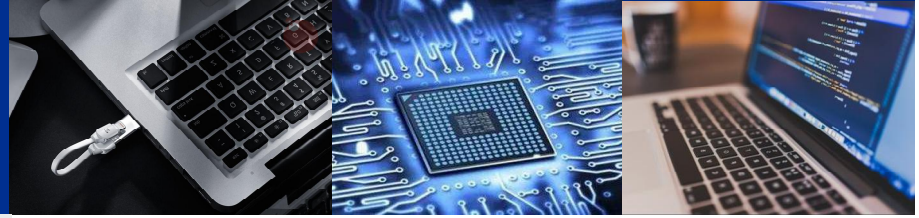
① 执行速度快。

② 程序短小。

③ 可以直接控制硬件。

④ 可以方便地编译。

⑤ 辅助计算机工作者掌握计算机体系结构。



4.1.3一般汇编语言程序的结构形式

```
stack   SEGMENT  PARA  'stack'
        DB   100  DUP('stack')
stack   ENDS
```

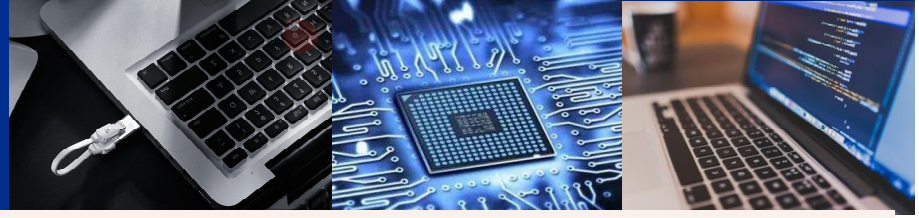
堆栈段

```
data    SEGMENT
        <数据、变量在此定义>
data    ENDS
```

数据段

```
code    SEGMENT
        ASSUME  CS:code, DS:data, ES:data, SS:stack
start:  MOV    AX, data
        MOV    DS, AX
        MOV    ES, AX
        <此处加入你自己的程序段>                    真指令
        MOV    AL, 4CH
        INT    21H
code    ENDS
        END    start
```

代码段



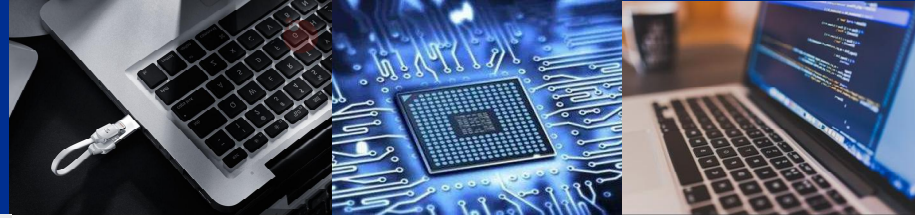
Masm for Windows 集成实验环境共享版 2015

文件(F) 编辑(E) 运行(R) 工具(O) 帮助(H)

新建 打开 保存 剪切 复制 粘贴 查找 注释 取注 撤销 恢复 运行 列表 调试 教程 日记 注册 网站 帮助

- 我的程序
  - 1.asm
- 软件应用问题解答
- 简单的入门程序实例
- 数据传送指令(免费)
- 算术运算指令
- 逻辑运算指令
- 串操作指令
- 程序转移指令
- 汇编伪指令
- DOS汇编源程序
- WINDOWS汇编实例源程序
- 汇编语言错误信息表
- DOS 功能调用表(免费)
- BIOS 功能调用表(免费)
- DEBUG 命令(免费)
- ASCII码字符表(免费)

```
01 DATAS SEGMENT
02     ;此处输入数据段代码
03 DATAS ENDS
04
05 STACKS SEGMENT
06     ;此处输入堆栈段代码
07 STACKS ENDS
08
09 CODES SEGMENT
10     ASSUME CS:CODES,DS:DATAS,SS:STACKS
11 START:
12     MOV AX,DATAS
13     MOV DS,AX
14     ;此处输入代码段代码
15     MOV AH,4CH
16     INT 21H
17 CODES ENDS
18     END START
```



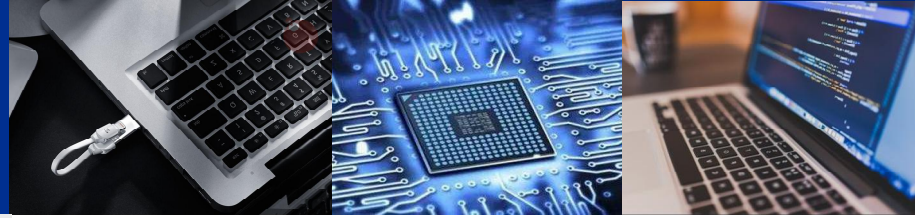
4.1汇编语言程序基本格式

每一段的语句体由语句序列组成，8086汇编语言语句分为如下三类。

① **指令语句**：8086指令系统的指令形式，与机器指令一一对应。

② **伪指令语句**：又称管理语句。在汇编语言源程序的汇编过程中起作用，它是对汇编程序的命令语句，一般没有相应的目标代码。

③ **宏指令语句**：是宏汇编程序能识别的、预先定义的指令代码序列。一旦定义以后，宏指令就像一条指令一样，可以在源程序中被引用，其效果等同于引入一段代码序列。



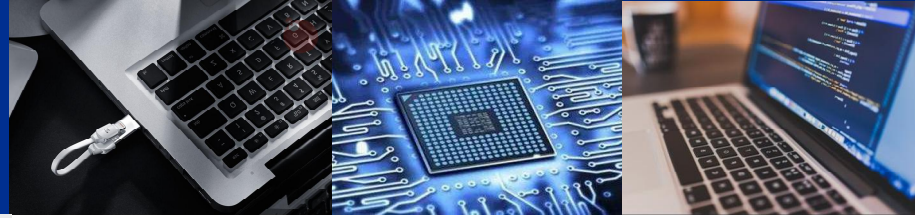
4.1汇编语言程序基本格式

一般情况下，汇编语言的**指令性语句**可以由以下几部分组成：

[标号:] [前缀] 操作码助记符 [操作数] [；注释]

伪指令语句又称为**指示性语句**，其格式为：

[名字] 伪指令定义符 [操作数] [； 注释]



4.2汇编语言中的数据

4.2.1常量

- **数字常量：** 可以是二进制数、八进制数、十进制数或十六进制数表示。

二进制：00011101B

十六进制：0C5AAH

十进制：9856D

八进制：237O

- **字符常量：** 是用单引号‘ ’括起来的ASCII字符，其值是该字符的ASCII代码值。

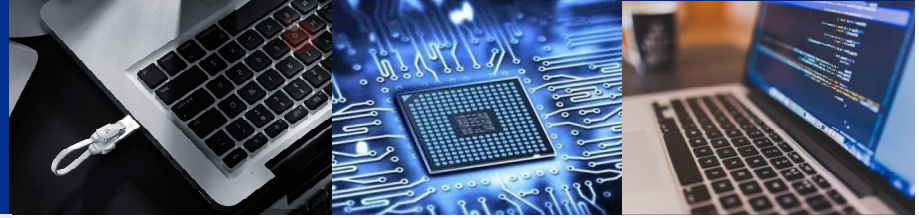
’The 2X means 2 multiple X’

- **符号常量：** 是用标识符定义的常量。

例如，采用符号定义语句 **PORTA EQU 80H**，

则指令 **MOV AL，PORTA**

与指令 **MOV AL，80H** 等价。



4.2汇编语言中的数据

4.2.2 变量

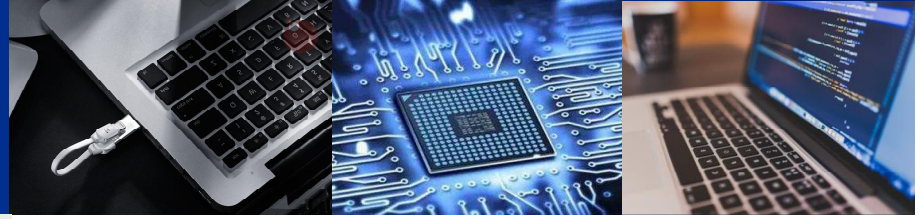
1. 数据定义伪指令

数据定义伪指令用来定义一个变量，并将所需要的数据放入指定的存储单元中，也可以只给变量分配存储单元，而不赋予特定的值。

- 数据定义伪指令的一般格式为：

  [变量名]　数据定义符　操作数 [，操作数…]　[；注释]

  - **常用的数据定义伪指令**
    - **定义字节变量伪指令　DB**
    - **定义字变量伪指令　DW**
    - **定义双字变量伪指令　DD**
    - **定义四字变量伪指令　DQ**
    - **定义十字节变量伪指令　DT**
    - **经过定义的变量有三个属性：数据类型、偏移量、段基址**



4.2汇编语言中的数据

表达式是给变量预置的初值，可以是下述情况之一：

①数值表达式：数值允许用二进制、八进制、十进制、十六进制形式书写。

②？：表示不预置确定的初值。

③字符串表达式：用引号括起来的不超过255个字符或其他ASCII码符号。DB伪指令将按顺序为字符串中每一个字符或符号分配一个字节单元，存放它们的ASCII编码，但DB以外的数据定义伪指令只允许定义最多两个字符的字符串，且按逆序存放在低地址开始的单元。

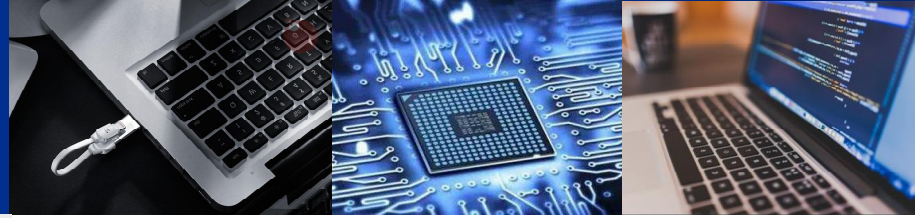
注意：3个及其以上的字符，只能用DB定义。当字符串的长度不超过两个字符时可用DW定义。

```
str1   DW   'abcd'
str2   DD   'abcd'
```



④带DUP操作符的表达式：DUP是定义重复数据操作符，它的使用格式是：

N DUP (Exp)；其中，N为重复次数，EXP为表达式。



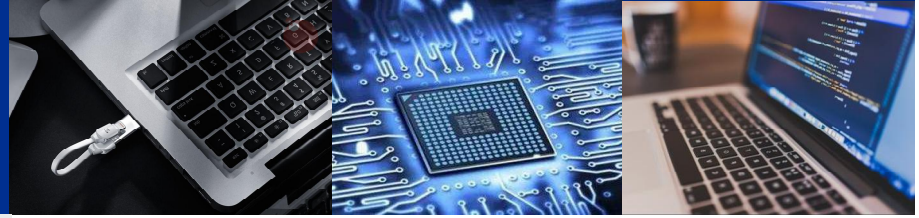
4.2汇编语言中的数据

例如：

```
DATA1   DB   25,25H,10011010B;数值表达式
DATA2   DB   ?,?;?表达式
DATA3   DB   2 DUP(2 DUP(4),15);DUP表达式
DATA4   DB   'AB','CD';字符串表达式
DATA5   DW   ?,?,-32768;字类型
```

DATA1	19H
	25H
	9AH
DATA2	?
	?
DATA3	04H
	04H
	0FH
	04H
	04H
	0FH

DATA4	42H
	41H
	44H
	43H
DATA5	?
	?
	?
	?
	00H
	80H



4.2汇编语言中的数据

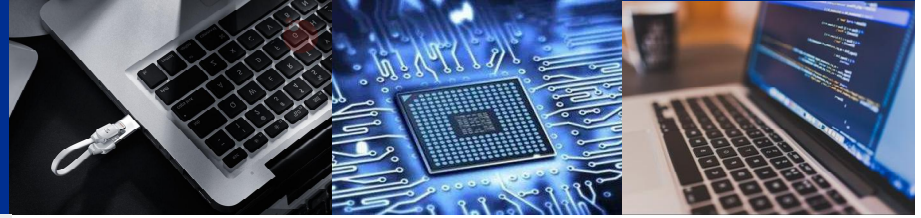
2.变量的属性

变量有三种属性：**段属性、偏移属性、类型属性。**

① **段属性**（SEGMENT）：变量所在的段的起始地址（16位），此值必须在一个段寄存器中。

② **偏移属性**（OFFSET）：该变量与段的起始地址之间相距的字节数。对于16位段，是16位无符号数; 对于32位段，则是32位无符号数。在当前段内给出的变量的偏移值等于当前地址计数器的值，当前地址计数器的值可以用$来表示。

③ **类型属性**（TYPE）：定义该变量的字节数。如BYTE（DB，1B长）、WORD（DW，2B长）、DWORD（DD，4B长）、FWORD（DF，6B长）、QWORD（DQ，8B长）、TBYTE（DT，10B长）。



4.2汇编语言中的数据

3.变量的使用

① 变量名作为存储单元的**直接地址**：变量名用直接寻址时, 变量的类型必须与指令的要求相符合。

例如: AB 已定义字节变量, AW 定义为字变量, 用变量名作直接寻址形式如下:

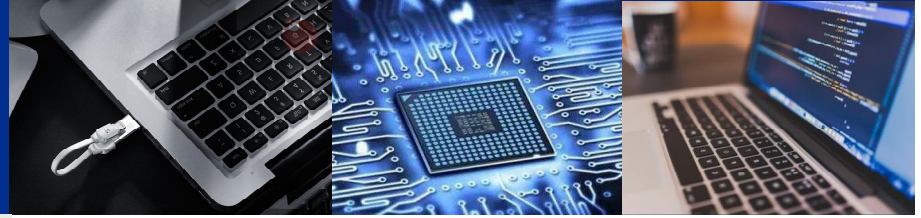
```
MOV   AH,AB
MOV   AX, AW
```

② 用合成运算符**PTR**可以临时改变变量类型。

例如，AB 已定义字节变量, AW 定义为字变量，指令序列如下:

```
MOV   CX, WORD PTR AB
MOV   CL, BYTE PTR AW
```

则临时把AB 变为字类型, AW 变为字节类型, 但段和偏移属性不变。



4.2汇编语言中的数据

③ 变量名作为相对寻址中的偏移量

例如，AB 已定义字节变量, AW 定义为字变量，在如下指令序列中：

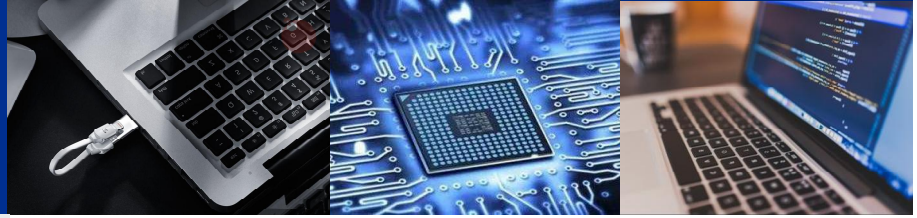
```
MOV  AX, AB[SI]
MOV  AX, AW[ BX][ SI]
```

AB, AW 分别表示它们的偏移量而不是它们所表示的数据，常用于数组或表格操作，AB[SI] 就表示AB 数组中第SI个元素。

④ 变量名仅对应数据区第一个数据项

例如:

```
WORD   DW 20 DUP( ?)
MOV    AX, WORD                    ; 第一个元素送AX,
MOV    AX, WORD+ 38                ; 第20 个元素送AX.
```



4.2汇编语言中的数据

4.2.3 标号

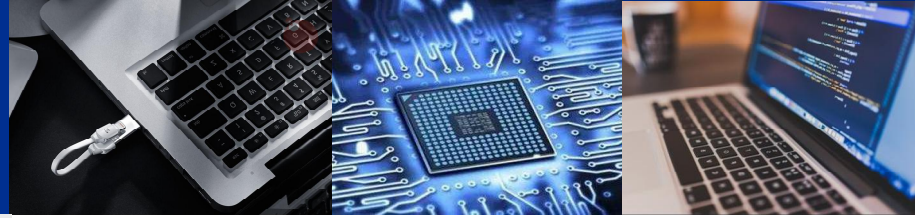
- 在指令性语句中，标号表示本条语句的符号地址；
- 标号、名字统称为标识符，都具备3种属性

——段属性、偏移属性及类型属性。

① 标号的段属性是定义标号在程序段的段地址。当程序中引用一个标号时，该标号的段值应在CS寄存器中。

② 标号的偏移量属性表示标号所在段的起始地址到定义该标号的地址之间的字节数。偏移量是一个16位无符号数。

③ 标号的类型属性有两种：NEAR和FAR。前一种标号可以在段内被引用，地址指针为2字节；后一种标号可以在其他段被引用，地址指针为4字节。



4.3运算符与表达式

表达式由操作数和运算符组成，如3+2、77 AND 55、NOT 5AH等。

1. 操作数

操作数用于代表一个数据或一个地址。

2. 运算符

运算符包括算术、逻辑、关系、分析、综合五类运算符。

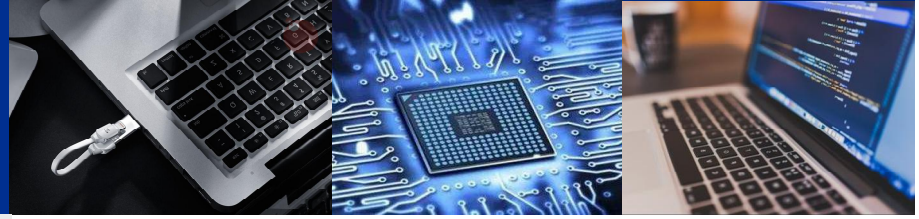
① 算术运算符 包括 +、 -、*、/ 、MOD

参加运算的内容可以是数据，也可以是地址，但在一个算术运算符的两边不能都是地址。

② 逻辑运算符 包括 AND、OR、NOT、XOR

③ 关系运算符 包括

相等：EQ	大于：GT
不等：NE	小于或等于：LE
小于：LT	大于或等于：GE



4.3运算符与表达式

关系运算结果只有两个——真、假。

如果关系是真，则结果为 0FFH（0FFFFH）；

如果关系是假，则结果为0。

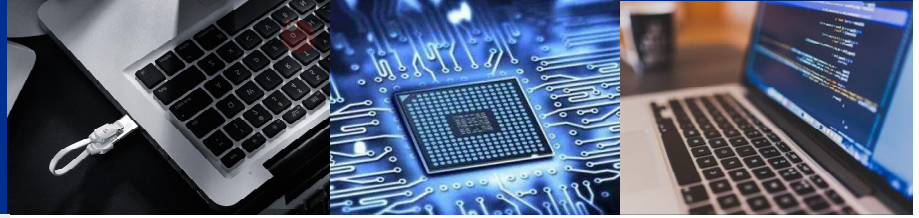
例如

MOV BX, DATA1 LT 0010H ═══ MOV BX, 0FFFFH

当 DATA1< 0010H 时

MOV BX, DATA1 LT 0010H ═══ MOV BX, 0

当 DATA1 > 0010H 时



4.3运算符与表达式

④取值运算符

取值运算在汇编语言程序设计中很重要。它可以把它存储器操作数分解成它的组成部分:

包括

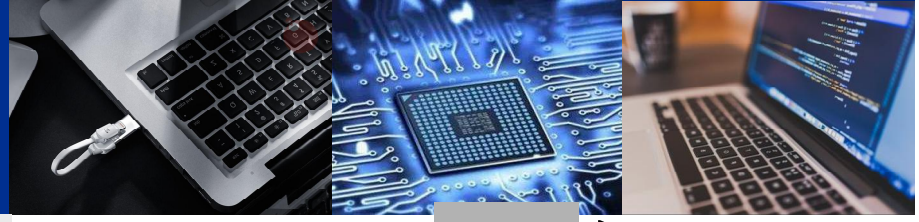
取存储单元偏移：OFFSET

取存储单元段： SEG

取类型：TYPE　TYPE：返回类型值：BYTE-1 WORD-2 DWORD-4 NEAR- -1 FAR- -2

取字节数： SIZE=LENGTH*TYPE

按类型取长度：LENGTH



4.3运算符与表达式

例1:设有数据段中使用

DATA1 DB ?

定义了一个单元，则

MOV AL, DATA1

可将DATA1中的数传送到AL中， AL=05

MOV AX, SEG DATA1

可将DATA1的所在段地址传到AX中，AX=2000

MOV AX, OFFSET DATA1

可将DATA1的偏移地址传入AX中， AX=1500

例2: 设有数据段中使用

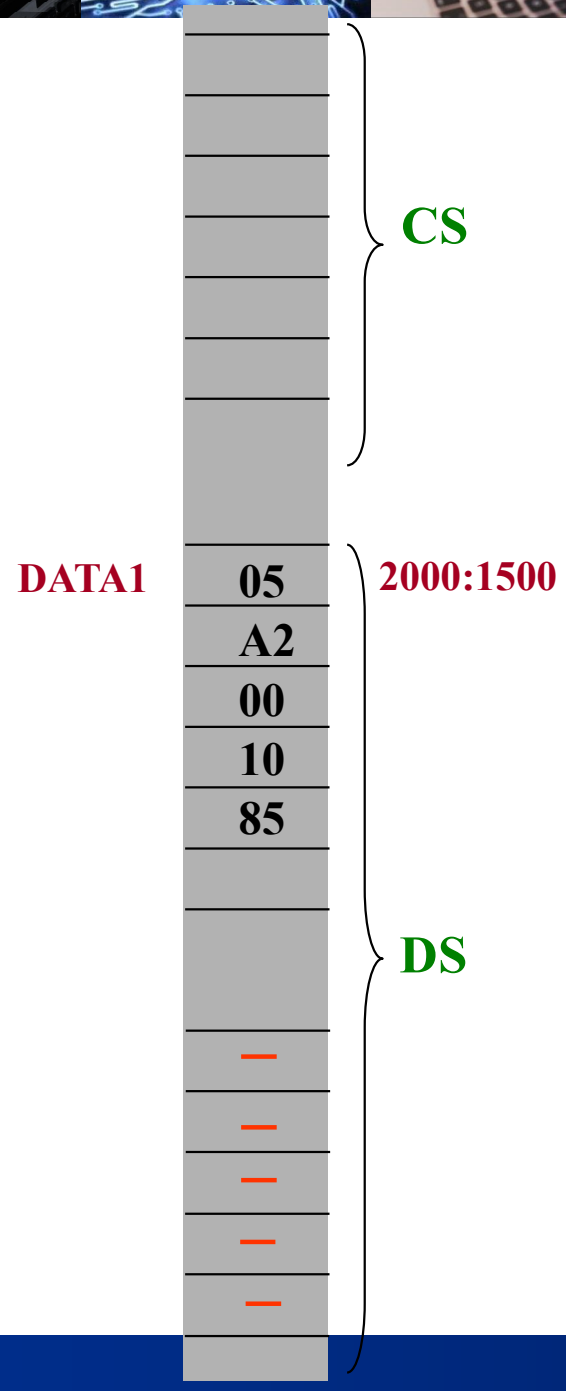
DATA1 DW 100 DUP (?)

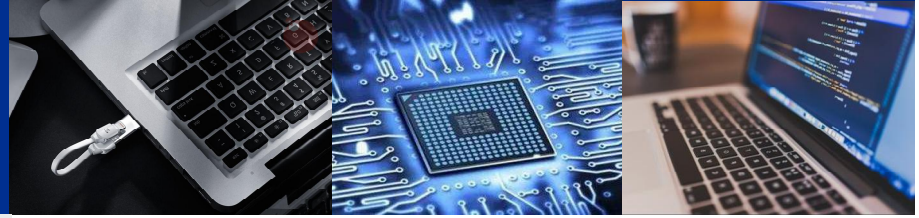
定义了100个字单元，则

MOV AX, TYPE DATA1 AX=2

MOV AX, SIZE DATA1 AX=200

MOV AX, LENGTH DATA1 AX=100





4.3运算符与表达式

⑤ 综合运算符

综合运算符包括两个地址操作符 PTR

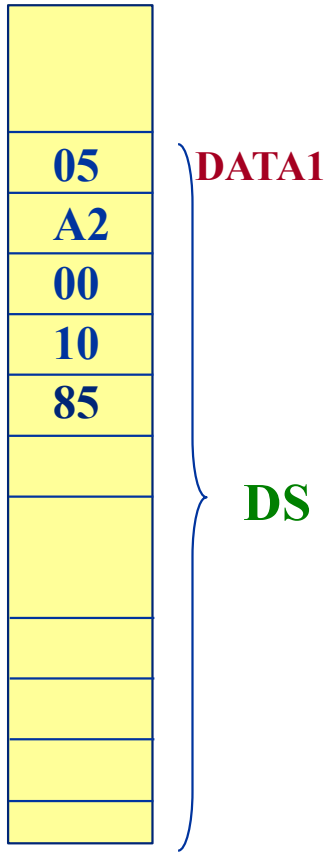
PTR——改变存储器地址操作数的操作类型，但其段地址和偏移地址不变。

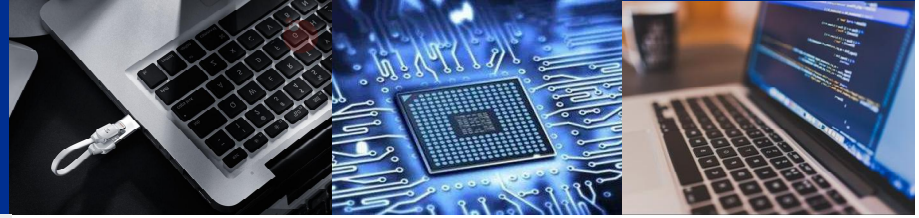
如：

DATA1 DW 100 DUP(?)

MOV AL, DATA1 ×

MOV AL, BYTE PTR DATA1 √





4.3运算符与表达式

在计算具有多个运算符的表达式值时，按以下规则操作：

◎优先级高的先运算，优先级低的后运算。

◎当优先级相同时，按表达式中从左到右的顺序进行运算

◎使用括号可以提高表达式运算的优先级。

◎表达式中的运算符的优先级

THIS :把运算符后面指出的类型属性赋给当前的存储单元

优先级		
高	1	括号中的项，即（…）和[…]
	2	LENGTH，SIZE，WIDTH，MASK
	3	PTR，OFFSET，SEG，TYPE，THIS
	4	×，/，MOD
	4	+，-
	6	EQ，NE，L，LE，GT，GE
	7	NOT
	8	AND
低	9	OR，XOR